

Linear Regression

Supervised Learning

Supervised learning is a type of machine learning where a model is trained on a set of labeled data, meaning each input is paired with the correct output. The model learns to predict the output for new input data by comparing its predictions with the actual answers provided in the training data, adjusting itself to minimize errors and improve accuracy.

Input

$$\{x_1, x_2, \dots, x_n\} \mid x_i \in \mathbb{R}^d$$

$$\{y_1, y_2, \dots, y_n\} \mid y_i \in \mathbb{R}$$

Regression

Given a dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ where $y_i \in \mathbb{R}$ learns a function

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

that captures the underlying relationship (pattern) between data-points and labels and generalizes it to unseen data-points. The defining characteristic of a regression problem is that the labels are continuous valued real numbers(Example - Rainfall Prediction)

Classification

Given a dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ where $y_i \in \{1, \dots, k\}$ learns a function

$$f : \mathbb{R}^d \rightarrow \{1, \dots, k\}$$

that captures the underlying relationship (pattern) between data-points and labels and generalizes it to unseen data-points. The defining characteristic of a classification problem is that the labels are from a finite, discrete set. If $k = 2$, the problem is called **binary classification** (Example - Spam vs Non-Spam mails). If $k > 2$, the problem is called **multi-class classification** (Example - Digit Classification)

Regression

Input

$$\{x_1, x_2, \dots, x_n\} \mid x_i \in \mathbb{R}^d$$

$$\{y_1, y_2, \dots, y_n\} \mid y_i \in \mathbb{R}$$

Goal of linear regression is to find a mapping between the input and output variables, represented as follows:

$$h : \mathbb{R}^d \rightarrow \mathbb{R}$$

- $\mathbb{R}^d$ is feature space
- $\mathbb{R}$ is label space

But the issue is, there are infinitely many function from $\mathbb{R}^d \rightarrow \mathbb{R}$, How do we measure the goodness of the function for respective dataset ?

$$error(h) = \sum_{i=1}^n (h(x_i) - y_i)^2$$

- $h(x_i) \rightarrow$ predicted label for x_i
- $y_i \rightarrow$ true label for x_i

Ideally, this error,

$$error(h) = \sum_{i=1}^n (h(x_i) - y_i)^2$$

should be minimized(0), which occurs when $h(x_i) = y_i$ for all i

However, achieving this may only result in memorizing the data and its outputs as this way we will get zero error on training data, which is not a desired outcome, as we care about the test performance.

To mitigate the memorization problem, introducing a structure to the mapping becomes necessary. The simplest and commonly used structure is linear, which we will adopt as the underlying structure for our data.

Simplest fix - Impose linear structure to reduce our search space

$$\mathcal{H}_{linear} = \left\{ h_w : \mathbb{R}^d \rightarrow \mathbb{R} \text{ s.t. } h(w) = w^T x \quad \forall w \in \mathbb{R}^d \right\}$$

Thus, our modified goal is to minimize:

$$\begin{aligned} \min_{h_w \in \mathcal{H}_{linear}} \sum_{i=1}^n (h(x_i) - y_i)^2 \\ \min_{h_w} \sum_{i=1}^n (w^T x_i - y_i)^2 \end{aligned}$$

Optimising The Error Function

$$\min_{w \in \mathbb{R}^d} \sum_{i=1}^n (w^T x_i - y_i)^2 = \|X^T w - y\|_2^2$$

$$X^T = \begin{bmatrix} \text{---} & x_1 & \text{---} \\ & \cdot & \\ & \cdot & \\ \text{---} & x_n & \text{---} \end{bmatrix}_{n \times d} \quad y = \begin{bmatrix} y_1 \\ \cdot \\ \cdot \\ y_n \end{bmatrix}_{n \times 1} \quad w = \begin{bmatrix} | \\ w \\ | \end{bmatrix}_{d \times 1}$$

Defining a function $f(w)$ that captures this minimization problem, we have:

$$\begin{aligned} f(w) &= \min_{w \in \mathbb{R}^d} (x^T w - y)^T (x^T w - y) \\ &= (x^T w - y)^2 \\ &= (x^T w)^2 - 2y(x^T w) + (y)^2 \\ &= w^T x x^T w - 2y x^T w + y^2 \end{aligned}$$

$$\nabla f(w) = 2(XX^T)w - 2(Xy)$$

Setting the gradient equation to zero $\nabla f(w) = 0$, we obtain:

$$2(XX^T)w - 2(Xy) = 0$$

$$2(XX^T)w = 2(Xy)$$

$$(XX^T)w = (Xy)$$

$$w^* = (XX^T)^{\dagger} (Xy)$$

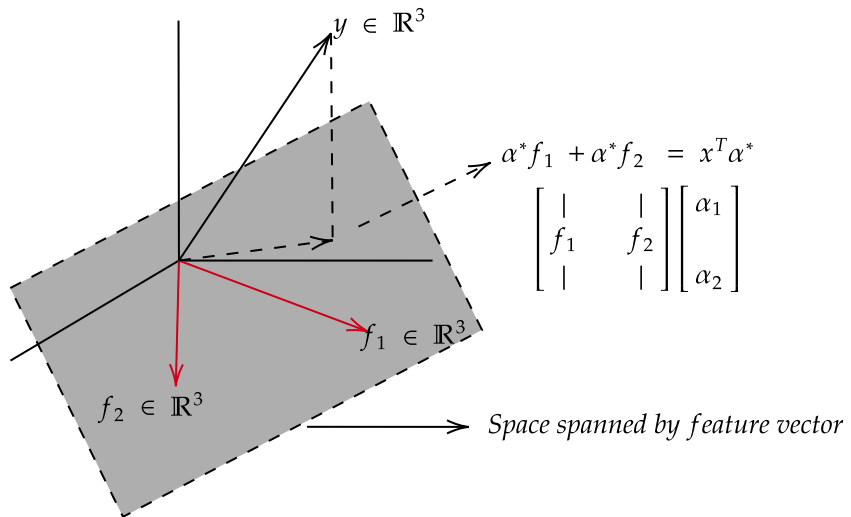
Here, $(XX^T)^{\dagger}$ represents the pseudo-inverse of (XX^T)

Geometric Interpretation

$$w^* = (XX^T)^{\dagger} (Xy)$$

for example, for any data matrix X of shape $d = 2, n = 3$

$$\begin{matrix} & f_1 & f_2 \\ \begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix} & \begin{bmatrix} | & | \\ | & | \\ | & | \end{bmatrix} & \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} \end{matrix}$$



$$\begin{aligned}
 (y - X^T \alpha^*)^T (y - X^T \alpha^*) &= 0 \\
 y^T x^T \alpha^* - \alpha^* (XX^T) \alpha^* &= 0
 \end{aligned} \tag{1}$$

We Know,

$$w^* = (XX^T)^\dagger (Xy)$$

Substituting $w^* = \alpha^*$ in eq. (1) we get,

$$y^T x^T \left((XX^T)^\dagger Xy \right) - (XX^T)^\dagger Xy (XX^T) \left((XX^T)^\dagger Xy \right) = 0$$

Conclusion, $X^T w^*$ corresponds to the projection of the labels onto the subspace spanned by the features.

Gradient Descent

The normal equation for linear regression $w^* = (XX^T)^{\dagger} (Xy)$ involves calculating $(XX^T)^{\dagger}$ which can be computationally expensive with a complexity of $O(d^3)$

But, We know w^* represents the solution of an unconstrained optimization problem, so taking the computational consideration into account it can be solved using gradient descent.

- Choose the initial point w^0
- Define the learning rate η
- Calculate the gradient at the current point $\nabla f(w^t)$
- Update parameters using the rule $w^{t+1} = w^t - \eta \nabla f(w^t)$

In our case, Sum of the squared error function is quadratic in nature $\left(\sum_{i=1}^n (w^T x_i - y_i)^2 \right)$ and belongs to convex function , thus it has unique global minima.

The iterative formula for gradient descent for linear regression :

$$\begin{aligned}w^{t+1} &= w^t - \eta^t \nabla f(w^t) \\w^{t+1} &= w^t - \eta^t [2(XX^T)w^t - 2(Xy)]\end{aligned}$$

Here, η is a scalar that controls the step-size of the descent, and t represents the current iteration.

Observe, there is no need to compute inverse but the issue arrives when $n \gg d$.

Stochastic Gradient Descent

for $t = 1, \dots, T$

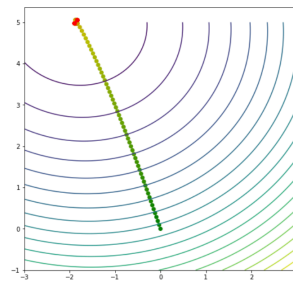
- At each step sample a bunch of datapoints (k) uniformly at random from set of all datapoints.
- Pretend that sample is the entire dataset and take gradient w.r.t. it

$$2(\tilde{X}\tilde{X}^T w^T - \tilde{X}\tilde{y}) \quad \tilde{X} \in \mathbb{R}^{d \times k}$$

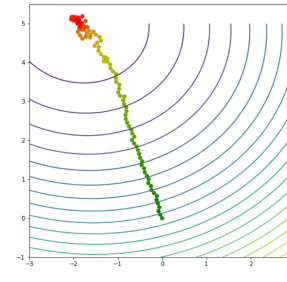
End

After T rounds we get,

$$w_{SGD}^T = \frac{1}{T} \sum_{i=1}^T w^i$$



Gradient Descent

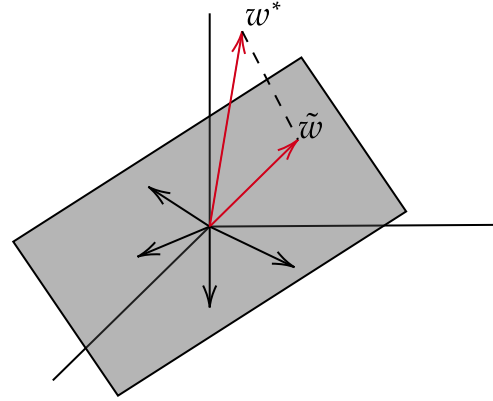


Stochastic Gradient Descent

The progress of SGD is less smooth given that we are only using an approximate version the gradient, but it is guaranteed to converge to optima with high probability.

Kernel Regression

$$w^* = (XX^T)^\dagger (Xy)$$



w^* must lie on the span of the data-points

$$\begin{aligned} w^* &= \tilde{w} + \tilde{w}_\perp \\ \forall i \quad w^{*T} x_i &= (\tilde{w} + \tilde{w}_\perp)^T x_i \\ &= \tilde{w}^T x_i + \tilde{w}_\perp^T x_i \\ &= \tilde{w}^T x_i \end{aligned}$$

$$\sum_{i=1}^n (w^T x_i - y_i)^2 = \sum_{i=1}^n (\tilde{w}^T x_i - y_i)^2$$

What if the data points reside in a non-linear subspace? Similar to dealing with non-linear data clustering, kernel functions are employed in this scenario as well

$$\begin{aligned}w^* &= X\alpha^* \text{ for some } \alpha^* \in \mathbb{R}^n \\X\alpha^* &= (XX^T)^\dagger (Xy) \\(XX^T)X\alpha^* &= (XX^T)(XX^T)^\dagger (Xy) \\(XX^T)X\alpha^* &= Xy \\X^T(XX^T)X\alpha^* &= X^TXy \\(X^TX)^2\alpha^* &= (X^TX)y \\K^2\alpha^* &= Ky \\ \alpha^* &= K^{-1}y\end{aligned}$$

To make prediction for some $X_{test} \in \mathbb{R}^d$

$$\begin{aligned} w^* &= \arg \min \sum_{i=1}^n (\phi(x_i) - y_i)^2 \\ &\quad \left(\sum_{i=1}^n \alpha^* \phi(x_i) \right)^T \phi(x_{test}) \\ &\quad \sum_{i=1}^n \alpha^* k(x_i, x_{test}) \end{aligned}$$

Here, α_i^* denotes the importance of the i -th data point in relation to w^* and $k(x_i, x_{test})$ signifies the similarity between x_{test} and x_i

ϕ transformation can be:

- Polynomial Kernel $(x^T y + 1)^p$
- RBF Kernel $\exp\left(\frac{-||x_i - x_{test}||_2^2}{2\sigma^2}\right)$

Probabilistic View of Linear Regression

Consider a dataset $\{(x_1, y_1) (x_2, y_2) \dots (x_n, y_n)\}$ with $x \in \mathbb{R}^d$; $y \in \mathbb{R}$

The probabilistic view of linear regression assumes that the target variable y_i can be modeled as a linear combination of the input features x_i , with an additional noise term ϵ following a zero-mean Gaussian distribution with variance σ^2 . Mathematically, this can be expressed as:

$$y_i = w^T x + \epsilon$$

Assume $\epsilon \sim \mathcal{N}(0, \sigma^2)$

$$\frac{y}{x} \sim \mathcal{N}(w^T x, \sigma^2)$$

To estimate the weight vector w that best fits the data, we can apply the principle of Maximum Likelihood (ML). The ML estimation seeks to find the parameter values that maximize the likelihood of observing the given data. Assuming that the noise term ϵ follows a zero-mean Gaussian distribution with variance σ^2 , we can express the likelihood function as:

$$\mathcal{L}(w, x_1, \dots, x_n, y_1, \dots, y_n) = \prod_{i=1}^n e^{\frac{-(w^T x_i - y_i)^2}{2\sigma^2}} \cdot \frac{1}{\sigma\sqrt{2\pi}}$$

$$\mathcal{L}(w, x_1, \dots, x_n, y_1, \dots, y_n) = \prod_{i=1}^n e^{\frac{-(w^T x_i - y_i)^2}{2\sigma^2}} \cdot \frac{1}{\sigma\sqrt{2\pi}}$$

$$\log(\mathcal{L}(w, x_1, \dots, x_n, y_1, \dots, y_n)) = \sum_{i=1}^n \frac{-(w^T x_i - y_i)^2}{2\sigma^2} \cdot \frac{1}{\sigma\sqrt{2\pi}}$$

$$\max_w \sum_{i=1}^n -(w^T x_i - y_i)^2 \equiv \min_w (w^T x_i - y_i)^2$$

$$\hat{w}_{ml} \equiv w^* = (XX^T)^{\dagger} (Xy)$$

This expression is equivalent to the mean squared error (MSE) objective function used in linear regression. Therefore, finding the maximum likelihood estimate $\hat{w}_{ml}$ assuming zero mean Gaussian noise is equivalent to solving the linear regression problem using the squared error loss.

Reference

Arun Raj Kumar, Lecture Slides for Machine Learning Techniques

Karthik Thiagarajan, Machine Learning Techniques Github Notes.

Indian Institute of Technology Madras , CS2007 Course Website

Stanford University, CS229 Machine Learning Notes

Thank You !